

Конспект: Support Vector Machines — Обнаружение мошенничества с вином

Постановка задачи

Датасет: wine_fraud.csv — 6497 записей о вине (красное и белое) с химическими характеристиками.

Цель: по 11 химическим признакам (кислотность, pH, алкоголь и др.) + типу вина определить, является ли бутылка легальной (Legit) или поддельной (Fraud).

1. Исследование данных (EDA)

Признаки датасета

Признак	Описание
fixed acidity	Фиксированная кислотность
volatile acidity	Летучая кислотность
citric acid	Лимонная кислота
residual sugar	Остаточный сахар
chlorides	Хлориды
free/total sulfur dioxide	Диоксид серы
density	Плотность
pH	Уровень кислотности
sulphates	Сульфаты
alcohol	Алкоголь
type	Тип вина: red / white

Признак	Описание
quality	Целевая переменная: Legit / Fraud

Дисбаланс классов — ключевая проблема

Legit	6251	(96.2%)
Fraud	246	(3.8%)

Это **несбалансированная задача** (imbalanced classification). Accuracy в 96% ничего не значит — модель может просто предсказывать "всё Legit".

Ключевые наблюдения из EDA

- **Fraud среди красных вин** встречается чаще относительно их доли, чем среди белых.
- **volatile acidity** и **chlorides** — наибольшая положительная корреляция с Fraud: у поддельных вин чаще завышены эти показатели.
- **alcohol** — отрицательная корреляция с Fraud: легальные вина в среднем крепче.

2. Предобработка данных

Шаг 1: Кодирование категориальных признаков

Столбец `type` (red/white) → dummy-переменная через `pd.get_dummies(drop_first=True)` :

```
df = pd.get_dummies(df, columns=["type"], drop_first=True)
# Появляется колонка type_white (0=red, 1=white)
```

Шаг 2: Разбивка данных

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.10, random_state=101
)
# 90% обучение, 10% тест
```

Шаг 3: Масштабирование — ОБЯЗАТЕЛЬНО для SVM

SVM очень чувствителен к масштабу признаков (использует евклидово расстояние).
StandardScaler приводит каждый признак к среднему=0, дисперсии=1.

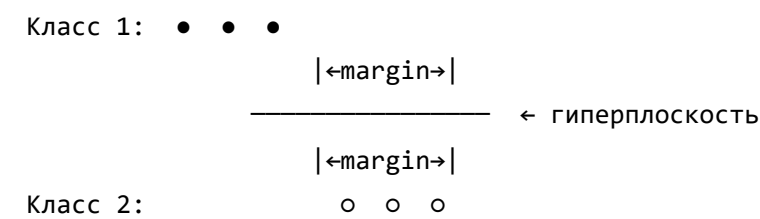
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit только на train!
X_test = scaler.transform(X_test)      # transform на test
```

Важно: fit_transform только на обучающей выборке, иначе будет "утечка данных" (data leakage).

3. Как работает SVM

Идея метода

SVM ищет **оптимальную разделяющую гиперплоскость** между классами с **максимальным зазором** (margin).



Опорные векторы (Support Vectors) — точки, которые ближе всего к гиперплоскости. Именно они определяют положение границы.

Kernel Trick — работа с нелинейными данными

Для нелинейно-разделимых данных SVM переводит признаки в пространство более высокой размерности через **ядровую функцию (kernel)**:

Kernel	Описание	Когда использовать
linear	Линейная граница	Простые задачи, много признаков
rbf	Гауссово ядро (по умолчанию)	Универсальный выбор

Kernel	Описание	Когда использовать
poly	Полиномиальное	Когда нужны степенные взаимодействия

Ключевые гиперпараметры

Параметр	Роль	Малое значение	Большое значение
C	Штраф за ошибки классификации	Мягкая граница, больше ошибок, модель проще	Жёсткая граница, меньше ошибок, риск переобучения
gamma	Влияние одной точки (для rbf)	Большой радиус влияния, плавная граница	Малый радиус, сложная "рваная" граница

4. Обучение и результаты

Базовая модель (параметры по умолчанию)

```
svc_base = SVC() # kernel='rbf', C=1.0, gamma='scale'  
svc_base.fit(X_train, y_train)
```

Результат:

	precision	recall	f1-score	support
Fraud	0.00	0.00	0.00	27
Legit	0.96	1.00	0.98	623
accuracy			0.96	650

Модель предсказала **0 случаев Fraud** — просто говорит "всё легально". Ассигасу=96%, но это бесполезно.

GridSearchCV — подбор гиперпараметров

```
param_grid = {  
    "C": [0.1, 1, 10, 100],  
    "gamma": [1, 0.1, 0.01, 0.001],  
    "kernel": ["rbf"]  
}  
  
grid = GridSearchCV(SVC(), param_grid, refit=True, cv=5, scoring="f1_macro")  
grid.fit(X_train, y_train)
```

Лучшие параметры: C=100, gamma=0.1

CV score (f1_macro): 0.6162

Лучшая модель

	precision	recall	f1-score	support
Fraud	0.29	0.07	0.12	27
Legit	0.96	0.99	0.98	623
accuracy			0.95	650

Confusion Matrix:

Предсказано:	Legit	Fraud
Реально Legit:	617	6
Реально Fraud:	25	2

5. Анализ результатов

Почему модель плохо находит Fraud?

1. **Дисбаланс классов** (96% Legit) — SVM оптимизирует общую точность, а не recall для Fraud.
2. **Мало примеров Fraud** — 246 из 6497. SVM видит их как "шум".

Метрики для несбалансированных задач

Метрика	Формула	Что показывает
Precision	$TP / (TP + FP)$	Из найденных Fraud — сколько правда Fraud
Recall	$TP / (TP + FN)$	Из реальных Fraud — сколько нашли
F1-score	$2 \times P \times R / (P + R)$	Баланс между precision и recall
f1_macro	среднее F1 по всем классам	Учитывает оба класса равнозначно

Для задачи обнаружения мошенничества **recall важнее precision** — лучше ложная тревога, чем пропущенный мошенник.

Как можно улучшить?

- `class_weight='balanced'` в SVC — автоматически увеличивает вес редкого класса
- SMOTE — синтетическая генерация примеров для меньшего класса
- Другие алгоритмы: случайный лес с `class_weight='balanced'`, XGBoost

6. Итоговая схема пайплайна

Данные → EDA → `get_dummies` → `train_test_split` → `StandardScaler`
→ SVC (default) → оценка → `GridSearchCV` → лучшая модель → оценка

7. Ключевые выводы

1. SVM — мощный алгоритм для задач классификации, особенно при чётком разделении классов и нормализованных данных.
2. **Масштабирование данных обязательно** для SVM.
3. На несбалансированных датасетах ассигасу вводит в заблуждение — используй `f1`, `recall`.
4. `GridSearchCV` подобрал `C=100`, `gamma=0.1`, но `recall` для Fraud всё равно остаётся низким (7%) из-за сильного дисбаланса.
5. Для реальных задач обнаружения мошенничества нужно дополнительно работать с дисбалансом классов.